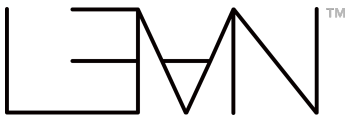


PROJECT PROPOSAL

KILT

Kripke in Lean Toolkit

A Verified LTL Model-Checking Toolkit in Lean



Yash Rawat

2023CS50334

COL876: Special Topics
in Formal Methods

September 2026

A LINEAR-TIME REQUIREMENT

$\mathbf{G}(\text{request} \rightarrow \mathbf{F} \text{grant})$

Every request is eventually followed by a grant.

An example liveness property for an infinite execution.

A Lean development connecting LTL semantics, Büchi automata, finite-state verification,
and kernel-checked correctness proofs.

Motivation and Background

Linear Temporal Logic (LTL) describes properties of individual, potentially infinite executions of a transition system. Operators such as next (X), eventually (F), globally (G), and until (U) express safety and liveness requirements; for example, $G(\text{request} \rightarrow F \text{ grant})$ states that every request is eventually granted. LTL is widely used to specify reactive behaviour and is commonly checked by translating a property into an automaton over infinite words, then analysing its interaction with the system.

Existing Work and Project Gap

LeanLTL is already a public Lean 4 library and a peer-reviewed ITP 2025 paper. It supports LTL reasoning over finite and infinite traces, offers a syntax macro, and provides automation for proofs about temporal formulas [1]. Separately, CSLib includes automata-theory foundations for finite and infinite words, including Büchi automata, products, and omega-language results [2,3]. A second Lean 4 project, `LeanearTemporalLogic`, formalizes LTL syntax, semantics, and transition systems; its README lists checked LTL model checking and Büchi formalization as ambitious future goals and describes the project as work in progress [4]. `Veil` is another relevant Lean-native verification framework for transition systems; I will study its workflow and examples as groundwork, while treating it as a reference rather than a required dependency [5].

These projects establish substantial foundations, so the proposal does not claim that LTL or Büchi automata are new to Lean, or that no Lean LTL checker is under development. The intended contribution is narrower: a CSLib-native, reusable path from a bounded LTL fragment through Büchi acceptance and product emptiness to a verified model-checking result. I will use LeanLTL as prior work and a semantic comparison point, rather than simply wrapping it as the project implementation; a small formula layer will be introduced only as needed to drive the automata construction and CSLib APIs. Where feasible, I will prove that its satisfaction relation agrees with LeanLTL's trace semantics. During the initial audit I will inspect `LeanearTemporalLogic`'s current implementation and discuss overlap through Lean Zulip, then narrow the module-level contribution to a clearly distinct and useful CSLib extension.

Goals and Expected Deliverables

- Audit LeanLTL, CSLib's automata APIs, `LeanearTemporalLogic`, and `Veil`; consult the Lean community about overlap, then select a clearly bounded LTL fragment and design only the formula interface needed for CSLib translation. Prove a compatibility result with LeanLTL's trace semantics where feasible.
- Develop an LTL-to-Büchi (or generalized Büchi) construction for the selected fragment, reusing CSLib's automata interfaces where appropriate.
- Build the product of a finite Kripke structure with the property automaton and an executable emptiness check for accepting infinite runs.
- Prove the central correspondence between the checker and the declared trace semantics. Correctness of the semantic and automata-theoretic core will take priority over extensions.
- If the core is complete, add a lightweight Lean metaprogramming frontend (notation, macro, or command) to make properties and checks convenient to write. Proof-producing automation and an initial CTL* study are further directions, not required deliverables.
- Deliver organized Lean source, examples/tests (including representative safety and liveness properties), and a short write-up of the design, proofs, trust boundary, and limitations.

Division of labour. This is an individual project. I will undertake the formal design, implementation, correctness proofs, interface development, evaluation, and documentation.

Expected Milestones: 24 September–15 November

The first week is reserved for research and setup. The dated blocks below show the intended progression; the core semantic correspondence and verified model-checking result take priority over frontend polish, witness presentation, and any CTL* extension.

Workstream	24–30 Sep	1–7 Oct	8–14 Oct	15–21 Oct	22–31 Oct	1–7 Nov	8–15 Nov
Literature/API audit, Lean setup, scoped design	•						
LTL syntax/semantics and Kripke execution model		•					
CSLib Büchi interface and formula-to-automaton core			•				
Product construction and accepting-run emptiness				•			
Correctness theorem and verified examples					•		
Lean frontend, evaluation, and write-up							•
Optional CTL* feasibility study / extensions							

The exact supported fragment and reuse boundary will be fixed after the September audit. Witnesses and proof automation will be attempted only after the verified checker is working; CTL* is an exploratory extension if the core goals are complete.

Selected References

1. E. Vin, K. A. Miller, and D. J. Fremont, “LeanLTL: A Unifying Framework for Linear Temporal Logics in Lean,” *ITP 2025*, LIPIcs 352, Article 37. doi:10.4230/LIPIcs.ITP.2025.37; code: <https://github.com/UCSCFormalMethods/LeanLTL>.
2. Leanprover, *CSLib: The Lean Computer Science Library*, automata modules and contribution guide. <https://github.com/leanprover/cslib>.
3. C.-T. Chou, *Automata Theory in Lean*, results being ported to CSLib. <https://github.com/ctchou/AutomataTheory>.
4. M. Pawagi, *LeanearTemporalLogic*, Lean 4 LTL formalization (work in progress). <https://github.com/mrigankpawagi/LeanearTemporalLogic>.
5. Verse Lab, *Veil: A Framework for Automated and Interactive Verification of Transition Systems*. <https://github.com/verse-lab/veil>.